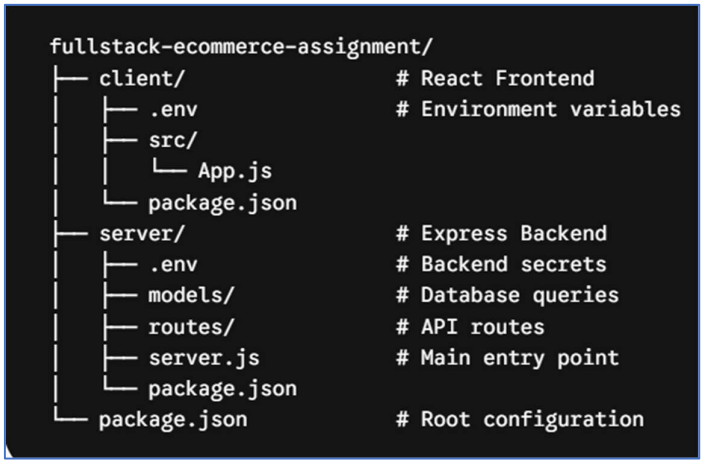
Pragya Singhal , CU24220050 **Assignment 1**

1. **Justification:** Direct importing between routes and models creates tight coupling and circular dependencies, violating the separation of concerns principle. Models should handle database logic independently while routes handle HTTP routing.



```
fullstack-ecommerce-assignment/
├── client/                 # React Frontend
│   ├── .env                # Environment variables
│   ├── src/
│   │   └── App.js
│   └── package.json
├── server/                 # Express Backend
│   ├── .env                # Backend secrets
│   ├── models/             # Database queries
│   ├── routes/             # API routes
│   ├── server.js           # Main entry point
│   └── package.json
└── package.json            # Root configuration
```

2. **Why it fails in production :** In production, the backend runs on a remote server/port rather than localhost:5000 on the user's machine. Hardcoding causes the browser to look locally on the client's device instead of querying your live backend server.

Rewritten Fetch Line using Environment Variable

// Create React App Standard

fetch(`${process.env.REACT_APP_API_URL}/api/products`)

// Vite + React Standard

fetch(`${import.meta.env.VITE_API_URL}/api/products`)

3. **.env file content:**

PORT=5000

DB_URL=mysql://root:password@localhost:3306/ecommerce_db

JWT_SECRET=supersecretkey123

Dotenv Loading Code (Node.js/Express):

require('dotenv').config();

const express = require('express');

const app = express();

const PORT = process.env.PORT || 5000;

Why not commit to Git (1 line): Committing .env exposes sensitive credentials publicly to your GitHub repository, creating a major security vulnerability.

4. **Express Code:**

```
const cors = require('cors');

app.use(cors());
```

Why the browser blocks it : The Same-Origin Policy prevents scripts loaded from one origin from interacting with resources from a different origin unless explicitly permitted by the server via CORS headers.

5.

Endpoint	Method	Request Body	Response
`/api/books`	`POST`	`{ "title": "String", "author": "String" }`	`201 Created` + JSON object of the created book
`/api/books`	`GET`	*None*	`200 OK` + JSON array of all books
`/api/books/:id`	`PUT`	`{ "title": "String", "author": "String" }`	`200 OK` + JSON object of the updated book
`/api/books/:id`	`DELETE`	*None*	`200 OK` or `204 No Content` (Success message)

6**. Refactored Code:**

```
const mysql = require('mysql2');

const db = mysql.createPool({

  host: 'localhost',

  user: 'root',

  database: 'shop'  });

app.get('/products', (req, res) => {

  db.query('SELECT * FROM products', (err, rows) => {

    if (err) return res.status(500).send(err);

    res.json(rows);

  });  });
```

Performance Problem : Creating a new connection on every request exhausts database connection limits and introduces severe latency. Establishing and tearing down TCP connections repeatedly wastes valuable CPU and memory overhead.

7: **Concurrently Full Stack Scripts**

package.json Scripts Section

```
 "scripts": {
```

"server": "nodemon server.js",

"client": "npm start --prefix client",

"dev": "concurrently \"npm run server\" \"npm run client\""

}

8. Frontend API Key Security Fix (Rewritten Code using Environment Variable)

const response = await fetch(

`https://api.weather.com/data?key=${process.env.REACT_APP_WEATHER_API_KEY}`

);

Security Risk : React code is compiled and publicly exposed in the browser, meaning anyone can view your hardcoded API key in the source. Malicious users can steal it to incur unauthorized costs or abuse rate limits.

9. **Steps:**

1. User enters credentials into the React form and clicks the submit button.

2. React captures form state and executes an API call (e.g., axios.post('/api/login', { username, password })).

3. Browser sends an HTTP POST request to the Express backend server.

4. Express backend route handler intercepts request and reads req.body.

5. Backend queries the MySQL database to locate user credentials (SELECT * FROM users WHERE username = ?).

6. Database returns the user record; backend securely compares credentials.

7. If valid, backend generates a JSON Web Token (JWT) and sends it back in the HTTP response.

8. React receives the response, stores the token, and updates the UI to redirect to the dashboard.

10. **Missing Line:** app.use(express.json());

Corrected Code:

JavaScript

const express = require('express');

const app = express();

app.use(express.json());

```
app.post('/login', (req, res) => {
  const { username, password } = req.body;
  res.send(`Welcome ${username}`);
});
app.listen(3000, () => console.log('Server running on port 3000'));
```

Missing Line Identification: The server is missing the built-in JSON body parsing middleware (app.use(express.json())). Without it, Express does not parse incoming JSON payloads, resulting in req.body being undefined.